Model Selection and Optimization + Prompt and Context Engineering

Domains 5 (16.8%) and 6 (11.0%) of the CCDV-F blueprint — 27.8% combined, the second-largest chunk of the exam after Applications and Integration.

Sources: Anthropic Partner Academy prep course (Module 1 — MSO Foundations; Module 2 — Production-Grade Prompting), cross-checked and extended against Claude Platform Docs: Models overview (https://platform.claude.com/docs/en/about-claude/models/overview), Extended thinking (https://platform.claude.com/docs/en/build-with-claude/extended-thinking), Prompt caching (https://platform.claude.com/docs/en/build-with-claude/prompt-caching), Effective context engineering for AI agents (https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents), and the Agent SDK: how the agent loop works (https://code.claude.com/docs/en/agent-sdk/agent-loop) (effort levels).

LLM Fundamentals (5.2%)

Tokens are the unit of everything. Claude doesn't read characters or words — it reads tokens, and the characters-per-token ratio depends on the model's tokenizer and changes across model generations (Fable 5, for instance, moved to the tokenizer introduced with Opus 4.7, which produces ~30% more tokens than pre-4.7 models for the same text). Never hardcode a chars-per-token constant — confirm the current tokenizer behavior at build time. Everything the model processes counts against tokens: prompt, conversation history, tool definitions, tool results, and the response itself. Tokens are the unit of both pricing and the context-window budget.

The context window is a fixed, shared budget — system prompt, full conversation history, injected documents, every tool result, and the model's own output all draw from the same pool. Two distinct failure modes:

- Input already larger than the window → the request is **rejected with a validation error before generation starts**.
- Input fits, but generation itself hits the ceiling mid-response → current models **stop and return partial output** with stop reason `model_context_window_exceeded`, not an error.

Either way, a long-running session needs the application to trim or summarize history before each call — this rarely bites in development (short test inputs) and reliably bites in production (longer inputs, more turns). See **Context Engineering** below for the mitigation.

Sampling and non-determinism. The model doesn't pick one fixed next token — at each step it produces a probability distribution and samples from it. `temperature` reshapes that distribution: lower concentrates probability on the most likely tokens (more repeatable), higher spreads it out (more varied). Because the choice is sampled, the same prompt run twice can return different (but equally correct) wording. **This is model-dependent**: the newest Claude models (Fable 5, Opus 5, Sonnet 5) do not accept non-default sampling parameters — setting `temperature`, `top_p`, or `top_k` returns a **400 error**, and output is steered through prompting instead. Even where `temperature` is accepted, `temperature: 0` makes output more repeatable but does **not** guarantee identical output across calls. Always confirm current parameter support in the API reference at build time rather than assuming it carries over from an older model generation.

Non-determinism is why exact-text assertions in tests are flaky: the model can express the same correct answer many different ways. Assert on the *property* that must hold (a required field is present, a value is in range, the output parses) instead of exact string matching; use a model-graded eval when you need to judge meaning rather than structure.

Prompting modes (shot count) — separate axis from wording style:

- **Zero-shot**: instruction only, no examples.
- **One-shot**: one input/output example.
- **Multi-shot / few-shot**: several examples.

Examples aren't training data — they live in the prompt and show the model the exact shape of the answer, which a description alone often can't pin down. Each example costs tokens on every call, so this is a quality/cost tradeoff: reach for zero-shot when the task and output shape are obvious, move to one/multi-shot when the output has a specific structure, casing, or edge case a description keeps missing. A more capable model often succeeds zero-shot where a smaller model needs a few examples — so adding examples can let a cheaper model do the job. Try the simplest model and fewest examples that meet your eval bar; add capability or examples only where the eval shows you need them.

Technical Fundamentals (6.1%)

SDK vs. raw REST. Claude is reached over an HTTP REST API — your code POSTs a JSON body to an endpoint with your API key and reads a JSON response. You can call it directly with any HTTP client, or use an official SDK (Python, TypeScript, and others), which is a thin convenience layer over the same REST API handling auth, request construction, retries, and response parsing. SDK and raw REST hit the same API and the same model — the SDK just removes boilerplate.

Synchronous vs. streaming vs. async vs. batch — four different problems, four different tools:

- **Synchronous**: send the request, wait for the full response, then act. Fine for short responses and backend jobs with no one watching.
- **Streaming**: the response comes back in pieces as the model generates it, over the same HTTP connection via **server-sent events**. Use when a response is long or a user is watching — output appears immediately instead of after a blank-screen wait; your code reassembles the pieces.
- **Async client**: the Python SDK exposes `AsyncAnthropic` for non-blocking `async` / `await` calls that don't tie up your application thread. The TypeScript SDK's standard client is already Promise-based — there's no separate async client class, you just `await` calls. Either way the request still returns in real time; async buys you concurrency, not lower latency or cost.
- **Message Batches API**: a separate pattern for bulk, offline, latency-tolerant work. Submit a large set of requests in one call, get an identifier back, poll for completion. Batch jobs can take **up to 24 hours** and run at a **lower per-token cost** in exchange for that latency — the right choice for offline pipelines, eval runs, and bulk jobs where no user is waiting and cost matters more than turnaround time.

```
# Sample 1 from the official exam guide is exactly this tradeoff:
# 10,000 documents, overnight, cost-sensitive, no urgency → Message Batches API,
# not synchronous calls in parallel and not just shrinking max_tokens.
```

Model Selection and Tradeoffs (2.7%)

Current lineup (verify against Models overview (https://platform.claude.com/docs/en/about-claude/models/overview) before relying on exact numbers — pricing and context windows shift release to release):

Tier	API ID	Positioning	Context window	Reasoning mode	Comparative latency
Fable 5	`claude-fable-5`	Most capable widely-released model; long-running agents	1M tokens	Adaptive thinking, **always on**	Slower
Opus 5	`claude-opus-5`	Complex agentic coding, enterprise work	1M tokens	Adaptive thinking	Moderate

Tier	API ID	Positioning	Context window	Reasoning mode	Comparative latency
Sonnet 5	`claude-sonnet-5`	Best speed/intelligence balance; default for most production workloads	1M tokens	Adaptive thinking	Fast
Haiku 4.5	`claude-haiku-4-5`	Fastest, near-frontier intelligence, cheapest	200K tokens	**Extended thinking** (not adaptive)	Fastest

Two gotchas worth knowing cold for the exam:

1. **Extended thinking and adaptive thinking are not the same feature, and current top-tier models don't both support the same one.** Fable 5, Opus 5, and Sonnet 5 use **adaptive thinking** (the model decides when/how much to think; tuned via `effort`), while Haiku 4.5 supports **extended thinking** (`thinking.type: "enabled"`) instead. Don't assume "the flagship models support extended thinking" — check the per-model capability table.
2. **`effort` defaults changed across releases**: on Opus 4.8 it defaults to `"high"` everywhere (API, Claude Code, claude.ai); on Opus 5 and Sonnet 5 it defaults to `"high"` on the API and Claude Code specifically. Don't assume a fixed universal default — confirm per-model.

The practical default workflow (straight from the prep course): start with **Sonnet**, move up a tier only when an eval shows the current tier missing your quality bar, move down to **Haiku** only when an eval shows the quality drop is acceptable for the task. Model choice trades off cost, latency, and capability — and it's evaluated empirically, not by intuition. **Reaching for the most capable model by default, without an eval forcing the question, is called out explicitly as the most common and most expensive model-selection mistake in production** — the cost of a wrong answer belongs in the calculation too: a cheaper tier that introduces costly downstream errors isn't actually a saving.

Routing applies the same idea as workflow routing (above) to model choice: a cheap classification call reads a request-level signal (task type, input length, a difficulty label) and sends the bulk of traffic to a default tier while routing only the requests that need it to a larger (or smaller) model — you pay for extra capability only where an eval shows it's earned.

```
def route(request):
    kind = classify(request)              # cheap call
    if kind == "simple_lookup":
        return call(model="haiku")
    return call(model="sonnet")           # or "opus" for the hardest slice
```

Skip the router and pin a single model when traffic is uniform in shape — the classification call only pays for itself when the traffic mix is genuinely heterogeneous.

Reasoning mode is a separate decision from model choice. On current models the reasoning mode is **adaptive thinking**: the model decides when and how much to think, tuned via an `effort` setting rather than a fixed token budget — the older `budget_tokens` control is **deprecated** and, on the newest model generations, **returns a 400 error**. Thinking content is omitted from responses by default on the newest models; request summarized display explicitly if you need to show it. Reasoning earns its cost on hard, multi-step problems and is wasted on lookups/classification.

`effort` levels (from the Agent SDK, same concept applies at the raw API level):

Level	Behavior	Good for
`low`	Minimal reasoning, fast	File lookups, listing
`medium`	Balanced	Routine edits, standard tasks
`high`	Thorough analysis	Refactors, debugging

Level	Behavior	Good for
`xhigh`	Extended reasoning depth	Complex coding/agentic tasks
`max`	Maximum depth	Multi-step problems needing deep analysis

Breaking changes across model releases: pinned model IDs (e.g. `claude-opus-4-8`) don't silently change behavior, but *migrating* to a new pinned ID can — sampling-parameter support, thinking mode, and default `effort` have all changed across recent generations. Read the migration guide before bumping a production pin.

Cost and Token Management (2.8%)

Prompt caching cuts cost and latency by reusing a previously-processed prompt prefix instead of reprocessing it:

- **Automatic caching**: one top-level `cache_control` field; the system applies the breakpoint to the last cacheable block and slides it forward as the conversation grows.
- **Explicit breakpoints**: `cache_control` on individual content blocks, up to **4 breakpoints per request**, subject to a 20-block lookback window and a minimum token threshold per block.
- **TTL choice**: **5-minute** default (resets on every read), or a **1-hour** TTL (`ttl: "1h"` on the breakpoint) that survives longer gaps. Pick 1-hour for bursty-but-infrequent traffic against the same prefix (e.g. an agent that pauses between steps); 5-minute for tight, frequent request loops. A prefix reused once an hour never earns back its write cost under the 5-minute default — the cache will have already expired.
- **Pricing (confirm current numbers at build time, these shift by release)**: cache **writes** are billed at a premium over base input tokens — roughly **1.25x for the 5-minute TTL, 2x for the 1-hour TTL**; cache **reads** cost a fraction of standard input, roughly **0.1x**. The economics only work when reads outnumber writes — a prefix hit once and never reused is a pure loss.
- Content that's stable across turns (system prompt, tool definitions, CLAUDE.md, a large tool schema) is exactly what you want cached — see Agent SDK: the context window (https://code.claude.com/docs/en/agent-sdk/agent-loop#the-context-window). One tradeoff worth naming: a cached prefix is assumed correct on the later request — if it needs to reflect data that can change, the cache can serve a stale version for as long as it lives. Fine for a fixed system prompt; not fine for content the use case needs live.

Token usage tracking and cost modeling: every response includes usage/cost fields (`total_cost_usd`, `usage` in the Agent SDK's `ResultMessage`; `usage` block in the raw Messages API). Track cache read vs. write vs. regular input tokens separately — they're priced differently, and a naive "total input tokens" number will misstate cost once caching is in play.

Context Engineering (3.8%)

Context engineering is the superset of prompt engineering: prompt engineering is about *what you say*; context engineering is about *curating the full set of tokens* — prompt, history, tool definitions, tool outputs, injected documents — available at each inference call, since context is a finite, shared resource.

Two distinct techniques for keeping a long session inside budget, and they solve different problems:

- **Tool output pruning / clearing**: if the bloat is mostly re-fetchable tool output (a big file read, a verbose command output), clear it once its immediate use has passed. This is lossless — the agent can just re-call the tool if it needs that content again — and cheap (no LLM call required to do it).
- **Compaction**: if the bloat is dialogue and reasoning that *can't* be cheaply re-fetched, summarize older history into a condensed form via an LLM call, keeping recent exchanges and key decisions intact. More expensive than pruning, but preserves non-recoverable context. The Agent SDK fires automatically as the

window approaches its limit, emitting a `compact_boundary` system event; a `PreCompact` hook can archive the full transcript first, and persistent rules belong in CLAUDE.md (re-injected every request) rather than the initial prompt, since compaction can lose specific early instructions.

Context isolation via subagents: each subagent starts with a fresh context window — no parent history, no accumulated tool outputs — and only its *final* response returns to the parent as a tool result. This is the cleanest way to keep the main agent's context lean while still doing deep, exploratory sub-work (see `1_agents_and_workflows.md` for the full mechanics).

Gotcha (real incident pattern — measure against production data, not dev fixtures): a team capped an agent's context at 40K tokens as a deliberate cost control (well under the model's actual 200K–1M ceiling). Development test fixtures averaged ~800 tokens per tool result; a full 20-turn session used ~18K tokens, comfortably inside budget. In production, real inputs carried more supporting material — tool output averaged ~3,200 tokens per call, and the budget cap was hit at **turn 8**, well before the task completed. The failure *looked* like degraded tool selection (the agent started choosing wrong tools, returning incomplete analyses) — the actual cause was that accumulated, never-pruned tool outputs had crowded out the system prompt and early instructions that told the agent which tool to use next. **The symptom of context overflow is frequently misread as a tool-selection or schema problem** — if tool selection degrades after a consistent number of turns, check whether the window is filling before debugging the schema. The fix: measure the token cost of a tool result against your *largest realistic production input*, not the shortest dev fixture, before shipping — and prune/compact proactively rather than waiting to hit the cap mid-task.

Prompt Engineering (4.6%)

A prompt that works once in interactive use often breaks in production against untested inputs. The fix is not "add more words" — it's diagnosing which *structural* piece is missing and adding exactly that piece. Rewording changes how you say something; it doesn't add a missing technique. If a prompt keeps getting longer with every iteration and still fails, that's the signal you're skipping diagnosis and just padding text.

The four techniques, and the failure signature that tells you which one is missing:

Symptom observed	Missing technique	Why it fixes it
Output comes back in the wrong *shape* (prose where you wanted a label, unstructured text where you wanted JSON)	**Output constraint**	The prompt never specified the form, field names, or stopping point
Content is off — scope drifts, tone shifts, Claude answers a broader question, gets worse deeper into the conversation	**System prompt** (or a more specific one)	The behavioral contract was too vague to hold across turns
Task is right but the structure is invented — Claude did the task but in a shape you never specified	**Few-shot examples**	A description alone can't pin down an exact structure; an example shows it
Clean on tested inputs, breaks on a variant/edge case	**A constraint covering that variant**	The prompt was validated against a narrow input set with no rule for the case that breaks it

Worked pattern (classification prompt, matches the exam guide's own sample-question style): a bare instruction ("classify the ticket") returns `"Billing"`, `"billing"`, and full sentences interchangeably — the downstream router expects one fixed label and breaks. The fix stacks three techniques together, because each does a job the others can't:

- **System prompt** sets the output contract (exactly one label from a fixed set, no other text).
- **XML tags** mark where each few-shot example starts/ends, so Claude doesn't read the examples as part of the live instruction.
- **Few-shot pairs** show the exact casing/format Claude should return, rather than describing it.

```
System: "You are a support classifier. Classify each ticket into exactly one
of: BILLING, TECHNICAL, ESCALATION. Return only the label. No other text."

<sample_input>My account shows two charges for April.</sample_input>
<ideal_output>BILLING</ideal_output>

<sample_input>The API keeps returning a 429 error.</sample_input>
<ideal_output>TECHNICAL</ideal_output>

User: <ticket>I was charged twice for the same month.</ticket>
```

When to stack vs. simplify: stack all four techniques against a clearly-defined output contract with edge cases few-shot can cover. Don't add all four to a task that only needs one — a "summarize this paragraph" prompt doesn't need an output schema and worked examples. If you've re-prompted five times and it's still wrong, stop and diagnose the failure type before adding more text.

System prompts carry the behavioral contract for the *whole session* — write once, treat as the persistent instruction layer (role, output format, rules that must not drift between turns). See `study_cheat_sheet.md` for input-sanitization guidance where system-prompt design intersects with prompt-injection defense (Domain 7).

Output Handling (2.6%)

Prompt-level output control (the four techniques above) is a *request*, not a guarantee — the model can still return a stray sentence, a wrong field name, or malformed JSON on an input you didn't test. **Structured outputs** move that guarantee from the prompt into the API itself via **constrained decoding**: you hand the API a JSON schema, and the model is constrained token-by-token to only emit tokens that keep the output valid against that schema — an invalid response literally cannot be generated.

Two distinct mechanisms, usable independently or together:

- **JSON outputs** (`output_config.format`, type `json_schema` + your schema): constrains the **final response**. Use when the model itself produces the structured payload your code consumes (extracting fields from a document, formatting an API response) — removes the parse-and-retry code you'd otherwise write around every call.
- **Strict tool use** (`strict: true` on a tool definition): constrains the **arguments Claude passes to your tools**, validated against the tool's input schema before your code runs. Use in agentic loops where a malformed tool argument would crash the function or trigger the wrong action.

Why this belongs in production code, not just the prompt: a prompt-level "return only JSON" instruction holds on the cases you tested and slips on the edge case you didn't — exactly the classification-prompt failure above. A schema constraint doesn't slip, because the API enforces it on every token instead of trusting the model to remember an instruction. This moves output correctness from "verified after the fact" to "ruled out before it happens."

Costs to weigh before enabling structured outputs everywhere:

- **First-request latency**: the API compiles your schema into a grammar before it can constrain output. Compiled grammars are cached for **24 hours from last use** — steady traffic on a stable schema pays the compile cost once; a workload that changes schemas constantly pays it repeatedly.
- **Token cost rises slightly**: the API injects a system prompt describing the expected format when structured outputs are on, and that's billed like any other input token.
- **A guaranteed schema is not a guaranteed success.** Two cases still don't parse: a **refusal** (`stop_reason: "refusal"` — the model declines for safety reasons) and a **truncation** (`stop_reason: "max_tokens"` — hits the output limit mid-structure). Always check `stop_reason` before assuming a response parses.

- **Incompatible with message prefilling.** JSON outputs and prefilling the assistant's response are mutually exclusive on the same request — pick whichever pattern fits the task.

Defensive parsing and skepticism toward confident output apply regardless of which mechanism you use: a fluent, confident-sounding response is not evidence of correctness. Validate structure (does it parse, are required fields present, are values in range) separately from validating meaning (which needs an eval with a model-graded judge, not a unit-test assertion — see Domain 4 in `5_eval_debugging_security.md`).